

- Description

This module manages a smart reservoir drainage and flood warning system, ensuring that:

1. **The water level in the reservoir is continuously monitored and maintained within safe limits.**
2. **If the water level exceeds the maximum safety threshold, controlled drainage is activated to prevent flooding.**
3. **If the water level drops below the minimum threshold, water conservation measures are implemented.**
4. **A log of all water level readings, drainage actions, and flood warnings is maintained for effective water management.**

This system helps optimize water storage, prevent floods, and ensure a stable water supply for agricultural and urban areas.

- Specification

module SmartReservoirManagement

var currentLevel, maxLevel, minLevel: real

var drainageActive: boolean

var log: sequence of string

process updateWaterLevel(changeAmount: real)

functional scenario form:

currentLevel <= maxLevel && log = log $\oplus$ ("Water Level Updated: " + currentLevel + "m, Status Normal."), Water level updated successfully.

||

currentLevel > maxLevel && drainageActive = true, log = log $\oplus$ ("Flood Risk Warning: " + currentLevel + "m, Activating Drainage."), Flood warning triggered, drainage activated.

||

currentLevel < minLevel && drainageActive = false, log = log $\oplus$ ("Water Shortage Alert: " + currentLevel + "m, Implementing Conservation Measures."), Water shortage alert triggered, conservation measures applied.

end process

end module

- Java Code

```
package code;
import java.util.*;
class Reservoir46 {
    private double maxLevel;
    private double minLevel;
    private double currentLevel;
    private boolean drainageActive;
    private List<String> log;
```

```

    public Reservoir46(double maxLevel, double minLevel, double initial
Level) {
        this.maxLevel = maxLevel;
        this.minLevel = minLevel;
        this.currentLevel = initialLevel;
        this.drainageActive = false;
        this.log = new ArrayList<>();
    }
    public void updateWaterLevel(double changeAmount) {
        currentLevel += changeAmount;
        manageDrainage();
        String message = String.format("Current Water Level: %.2f meter
s", currentLevel);
        if (currentLevel > maxLevel) {
            message += " | WARNING: Flood Risk! Activating Controlled D
rainage.";
        } else if (currentLevel < minLevel) {
            message += " | ALERT: Water Shortage Risk! Implementing Con
servation Measures.";
        } else {
            message += " | Water Level is Normal.";
        }
        System.out.println(message);
        log.add(message);
    }
    private void manageDrainage() {
        if (currentLevel > maxLevel) {
            drainageActive = true;
        } else {
            drainageActive = false;
        }
    }
    public void printLog() {
        System.out.println("\nReservoir Water Level Log:");
        for (String logEntry : log) {
            System.out.println(logEntry);
        }
    }
}
public class c46_SmartReservoirSystem {
    public static void main(String[] args) {
        Reservoir46 reservoir = new Reservoir46(50.0, 10.0, 30.0);
        // testcase-VT:
        //     reservoir.updateWaterLevel(15.0);
    }
}

```

```
//      reservoir.updateWaterLevel(-20.0);
//      reservoir.updateWaterLevel(-18.0);
//      reservoir.updateWaterLevel(20.0);
//      reservoir.updateWaterLevel(30.0);
//      reservoir.printLog();

//      // testcase-FT:
//      reservoir.updateWaterLevel(7.0220245063344835E305);
//      reservoir.updateWaterLevel(-1.7976931348623157E308);
//      reservoir.updateWaterLevel(4.582355049649039E307);
//      reservoir.updateWaterLevel(0.0);
//      reservoir.updateWaterLevel(2.7414542845469334E293);
//      reservoir.printLog();

//      // testcase-Z3:
//      reservoir.updateWaterLevel(15);
//      reservoir.updateWaterLevel(20);
//      reservoir.updateWaterLevel(18);
//      reservoir.updateWaterLevel(20);
//      reservoir.updateWaterLevel(30);
//      reservoir.printLog();

//      // testcase-UVT:
//      reservoir.updateWaterLevel(15);
//      reservoir.updateWaterLevel(-20);
//      reservoir.updateWaterLevel(-18);
//      reservoir.updateWaterLevel(20);
//      reservoir.updateWaterLevel(30);
//      reservoir.printLog();
    }
}
```

- Coverage report

1. Test case number: 5
2. V-Method testing branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	 1.0 %	8	834	842
▼ src	 1.0 %	8	834	842
▼ code	 1.0 %	8	834	842
▼ c46_SmartReservoirSystem.java	 100.0 %	8	0	8
> c46_SmartReservoirSystem		0	0	0
> Reservoir46	 100.0 %	8	0	8

3. Vibration testing instruction coverage: 97.7%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
✓ VibrationTestingTool	0.6 %	129	21,260	21,389
✓ src	0.6 %	129	21,260	21,389
✓ code	0.6 %	129	21,260	21,389
✓ c46_SmartReservoirSystem.java	97.7 %	129	3	132
> c46_SmartReservoirSystem	89.3 %	25	3	28
> Reservoir46	100.0 %	104	0	104

4. Fuzz testing branch coverage: 87.5%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
✓ VibrationTestingTool	0.8 %	7	835	842
✓ src	0.8 %	7	835	842
✓ code	0.8 %	7	835	842
✓ c46_SmartReservoirSystem.java	87.5 %	7	1	8
> c46_SmartReservoirSystem		0	0	0
> Reservoir46	87.5 %	7	1	8

5. Fuzz testing instruction coverage: 95.5%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
✓ VibrationTestingTool	0.6 %	126	21,263	21,389
✓ src	0.6 %	126	21,263	21,389
✓ code	0.6 %	126	21,263	21,389
✓ c46_SmartReservoirSystem.java	95.5 %	126	6	132
> c46_SmartReservoirSystem	89.3 %	25	3	28
> Reservoir46	97.1 %	101	3	104

6. Z3 branch coverage: 62.5%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
✓ VibrationTestingTool	0.6 %	5	835	840
✓ src	0.6 %	5	835	840
✓ code	0.6 %	5	835	840
✓ c46_SmartReservoirSystem.java	62.5 %	5	3	8
> Reservoir46	62.5 %	5	3	8
> c46_SmartReservoirSystem		0	0	0

7. Z3 instruction coverage: 80%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
✓ VibrationTestingTool	0.5 %	104	22,568	22,672
✓ src	0.5 %	104	22,568	22,672
✓ code	0.5 %	104	22,568	22,672
✓ c46_SmartReservoirSystem.java	80.0 %	104	26	130
> Reservoir46	77.9 %	81	23	104
> c46_SmartReservoirSystem	88.5 %	23	3	26

8. Unimproved V-method branch coverage: 75%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
✓ VibrationTestingTool	0.7 %	6	834	840
✓ src	0.7 %	6	834	840
✓ code	0.7 %	6	834	840
✓ c46_SmartReservoirSystem.java	75.0 %	6	2	8
> Reservoir46	75.0 %	6	2	8
> c46_SmartReservoirSystem		0	0	0

9. Unimproved V-method instruction coverage: 83.1%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
✓ VibrationTestingTool	0.5 %	108	22,564	22,672
✓ src	0.5 %	108	22,564	22,672
✓ code	0.5 %	108	22,564	22,672
✓ c46_SmartReservoirSystem.java	83.1 %	108	22	130
> Reservoir46	81.7 %	85	19	104
> c46_SmartReservoirSystem	88.5 %	23	3	26